

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Анализ набора данных Iris и демонстрация переобучения

Салех Али(2) ИУ1И-41М

Москва 2026 г

1. Цель работы

Цель лабораторной работы — освоить базовые понятия машинного обучения, выполнить визуальный анализ данных, построить модель бинарной классификации и на практическом примере показать явление переобучения.

Ссылка на Git-репозиторий: [<https://github.com/abomhiar590-cyber/ml-lab1.git>]

2. Теоретическая разминка

2.1. В чём состоит основная проблема переобучения?

Переобучение возникает в ситуации, когда модель слишком точно подстраивается под обучающую выборку и начинает учитывать не только закономерности, но и шум, выбросы и случайные особенности конкретных наблюдений. В результате качество на обучающих данных оказывается высоким, а на новых данных модель работает заметно хуже.

2.2. Почему нельзя оценивать качество модели на тех данных, на которых она обучалась?

Оценка на обучающей выборке показывает, насколько хорошо модель запомнила уже увиденные примеры, но не отражает её способность обобщать информацию. Такая оценка обычно чрезмерно оптимистична. Поэтому для честной проверки качества необходимо использовать отдельную тестовую выборку или процедуры кросс-валидации.

2.3. Что такое регуляризация и как она помогает бороться со сложностью модели?

Регуляризация — это способ ограничить сложность модели, добавив к функции потерь штраф за слишком большие коэффициенты или чрезмерную гибкость. Благодаря этому модель меньше подстраивается под шум, её параметры становятся устойчивее, а качество обобщения на новых данных улучшается.

3. Подготовка данных и загрузка iris.zip

Архив `iris.zip` содержит классический датасет Fisher Iris. Для решения задания использовался файл `iris.data`, включающий 150 объектов, 4 числовых признака и текстовую метку класса.

- Признаки: `sepal_length`, `sepal_width`, `petal_length`, `petal_width`.
- Классы: `Iris-setosa`, `Iris-versicolor`, `Iris-virginica`.
- Распределение классов сбалансировано: по 50 объектов в каждом классе.

Код 1. Загрузка данных из архива и формирование таблицы `DataFrame`.

```

from pathlib import Path
import zipfile
import pandas as pd

zip_path = Path("iris.zip")
extract_dir = Path("data")
extract_dir.mkdir(exist_ok=True)

with zipfile.ZipFile(zip_path, "r") as zf:
    zf.extractall(extract_dir)

columns = ["sepal length", "sepal width", "petal length", "petal width", "species"]
df = pd.read_csv(extract_dir / "iris.data", header=None, names=columns).dropna()
print(df.shape)
print(df["species"].value_counts())

```

Класс	Количество объектов
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

4. Визуальный анализ данных: pairplot

Для первичного исследования распределения признаков был построен `pairplot`, где по диагонали показаны гистограммы каждого признака, а вне диагонали — попарные диаграммы рассеяния.

Pairplot признаков датасета Iris

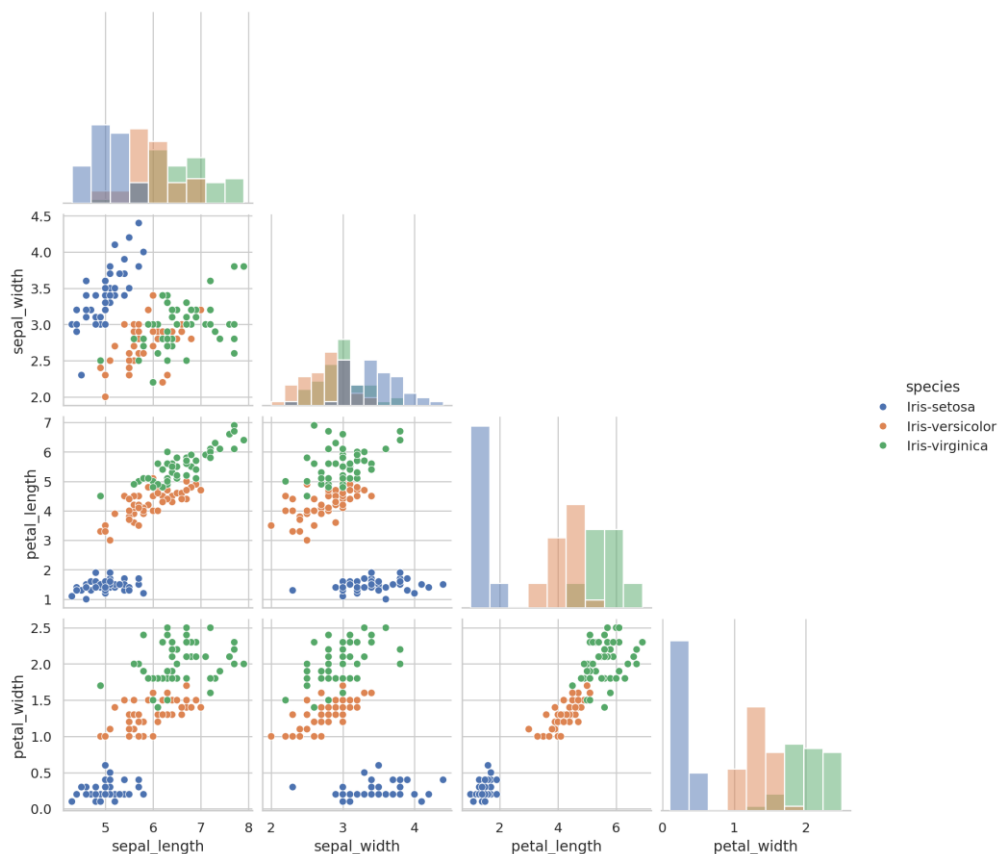


Рисунок 1. Pairplot признаков датасета Iris.

- Класс Iris-setosa визуально отделяется лучше всего и образует компактный кластер.

- Наиболее информативными признаками являются `petal_length` и `petal_width`.
- Классы `Iris-versicolor` и `Iris-virginica` частично пересекаются, поэтому задача классификации остаётся нетривиальной.

5. LogisticRegression для бинарной классификации

Для выполнения условия о бинарной классификации был выбран сценарий «`Iris-virginica` против всех остальных». Целевая переменная строится следующим образом: `1` — объект относится к `Iris-virginica`, `0` — объект относится к другим видам. Перед обучением выполняются стратифицированное разбиение на `train/test` и стандартизация признаков.

Код 2. Подготовка выборок и обучение логистической регрессии.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X = df.drop(columns=["species"])
y = (df["species"] == "Iris-virginica").astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

logreg_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])

logreg_pipeline.fit(X_train, y_train)
y_pred = logreg_pipeline.predict(X_test)
```

Полученные метрики качества на тестовой выборке:

Метрика	Значение
Accuracy	0.947
Precision	1.0
Recall	0.846
F1-score	0.917

Матрица ошибок модели:

Класс	Предсказано: не virginica	Предсказано: virginica
Истинно: не virginica	25	0
Истинно: virginica	2	11

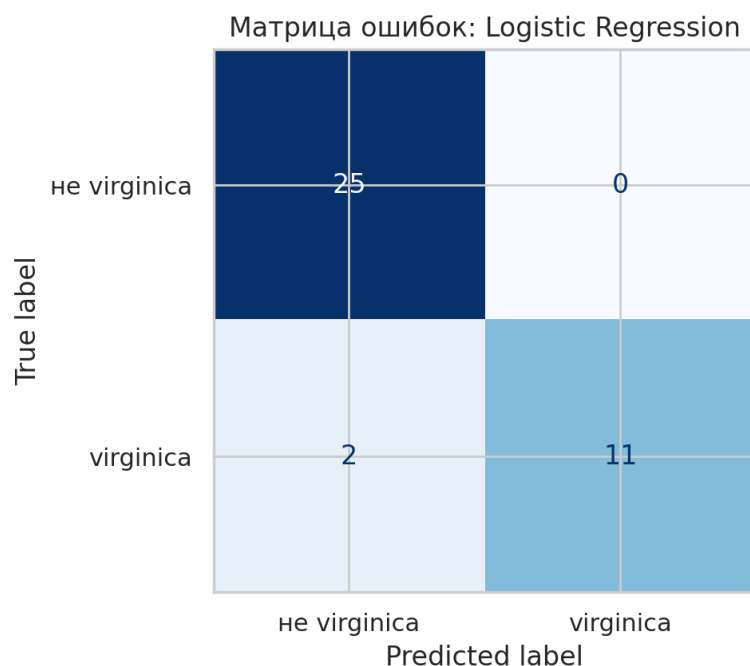


Рисунок 2. Матрица ошибок для модели Logistic Regression.

Вклад признаков в модель (по модулю коэффициента):

Признак	Коэффициент
petal_width	2.731
petal_length	1.983
sepal_length	0.529
sepal_width	-0.386

Интерпретация результата: модель достигает ассигуры около 0.95 и F1-score около 0.92, а наибольший вклад в решение дают признаки petal_width и petal_length. Это полностью согласуется с визуальным анализом pairplot.

6. Демонстрация переобучения модели линейной регрессии

Для наглядной демонстрации переобучения были сгенерированы синтетические данные по зависимости $y = \sin(2\pi x) + \text{шум}$. Далее использовалась линейная регрессия на полиномиальных признаках разных степеней. Формально модель остаётся линейной по параметрам, но её гибкость быстро растёт по мере увеличения степени полинома.

Код 3. Идея эксперимента по демонстрации переобучения.

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```

rng = np.random.default_rng(42)
X_syn = np.sort(rng.uniform(0, 1, size=30)).reshape(-1, 1)
y_syn = np.sin(2 * np.pi * X_syn.ravel()) + rng.normal(0, 0.15, size=30)

degrees = [1, 2, 3, 5, 7, 10, 15, 20]
for degree in degrees:
    model = Pipeline([
        ("poly", PolynomialFeatures(degree=degree, include_bias=False)),
        ("linreg", LinearRegression())
    ])
    model.fit(X_train_syn, y_train_syn)

```

Результаты эксперимента:

Степень полинома	Train MSE	Test MSE	Train R ²	Test R ²
1	0.1847	0.1738	0.6688	0.6102
2	0.1835	0.1644	0.6709	0.6313
3	0.0079	0.0385	0.9859	0.9137
5	0.0076	0.0333	0.9863	0.9254
7	0.0074	0.0276	0.9867	0.9382
10	0.0046	0.0205	0.9918	0.9541
15	9.831e-04	4.011e+02	0.9982	-8.985e+02
20	3.282e-05	6.225e+05	0.9999	-1.396e+06

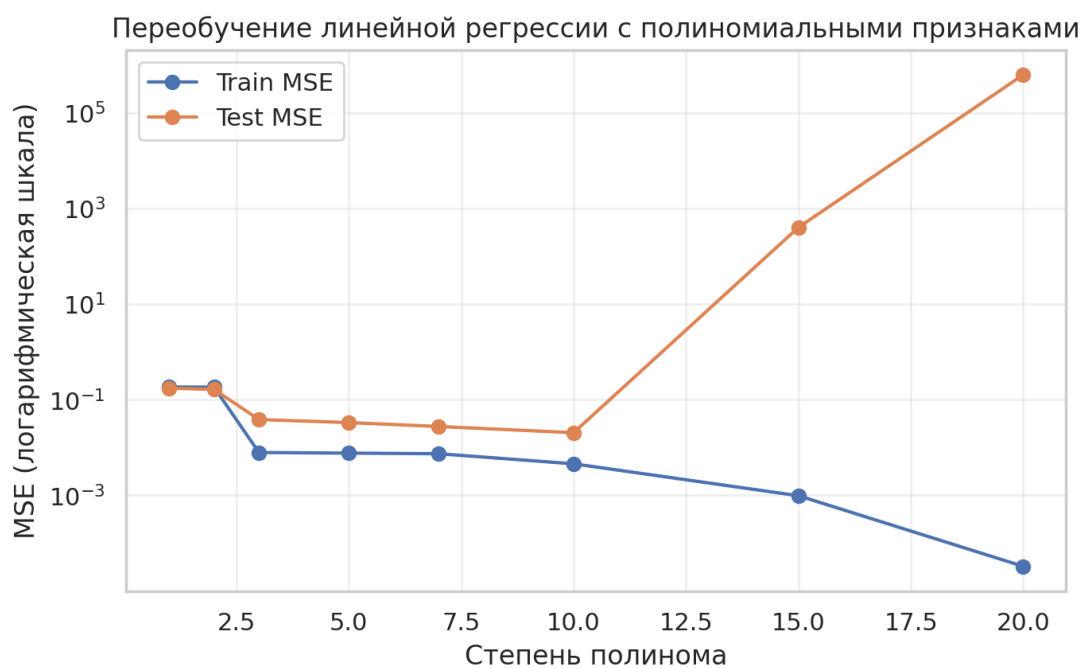


Рисунок 3. Зависимость ошибки train/test от степени полинома.

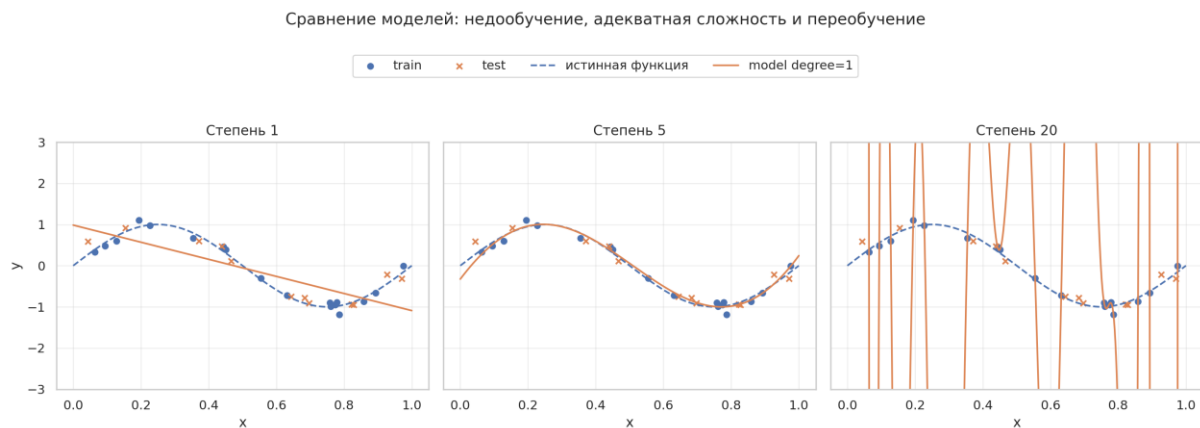


Рисунок 4. Сравнение недообучения, адекватной сложности и переобучения.

- При низкой степени полинома модель слишком проста и не улавливает реальную форму зависимости.
- При умеренной степени ошибки на тесте мала, а форма аппроксимации близка к истинной функции.
- При высокой степени полинома train MSE почти исчезает, но test MSE резко возрастает: модель запоминает шум и теряет способность к обобщению.

7. ВЫВОД

В ходе лабораторной работы были выполнены все задания из условия. Датасет Iris загружен из архива `iris.zip`, исследован визуально и использован для построения модели бинарной классификации на основе `LogisticRegression`. Модель показала высокое качество и подтвердила, что ключевыми признаками являются размеры лепестка.

На отдельном примере с синтетическими данными продемонстрировано переобучение линейной регрессии при чрезмерном усложнении полиномиальных признаков: ошибка на обучении снижается почти до нуля, а ошибка на тесте резко возрастает. Таким образом, цели лабораторной работы достигнуты полностью.

Приложение. Состав подготовленных файлов

- lab1_iris_analysis.ipynb — Jupyter Notebook с кодом и markdown-пояснениями.
- report_lab1.docx — итоговый отчёт в формате Word.
- figures/*.png — изображения, использованные в отчёте и notebook.